



Desafío de Programación: Gestión de Estudiantes

Este desafío de programación representa la última instancia de evaluación dentro del cursado de la asignatura.



Evaluación y Oportunidad de Promoción

Este Desafío de Programación, se estructura de la siguiente manera:

- Usted deberá responder a un cuestionario de autoevaluación diseñado para evaluar la comprensión y aplicación de los conceptos abordados en dicho material y desarrollo de la asignatura.
- Intentos de Aprobación: Se le concederán dos intentos máximos para la aprobación del desafío.
- Acceso a Promoción: El acceso a la promoción de la asignatura está supeditado a las siguientes condiciones, las cuales deben cumplirse simultáneamente:
 1. Obtener una calificación de 8 o superior en el cuestionario del desafío.
 2. Haber alcanzado la misma calificación (8 o superior) en los cuestionarios de autoevaluación anteriores de la asignatura y siempre en el primer intento para cada uno de ellos.



Consigna:

Desarrollar un programa que permita la Gestión de Información de Estudiantes. Este desafío tendrá dos implementaciones:



Requisitos Desafío - Implementación 1

En la primera implementación el programa utiliza estructuras de control, funciones y estructuras de datos internas para el manejo de la información en memoria RAM a través de un Menú Interactivo de opciones

1. Estructura de Datos para representar la información
2. Funciones: El código debe estar organizado en funciones para las siguientes operaciones:
 - Muestra las opciones del menú.
 - Permite dar de alta un nuevo estudiante.
 - Permite dar de baja un estudiante.
 - Permite modificar los datos de un estudiante existente.

- o Muestra un listado de estudiantes (o un mensaje si no hay estudiantes)
 - o La función principal que contiene el bucle del menú.
3. Estructuras de Control
 4. Menú interactivo de opciones



Requisitos Desafío - Implementación 2

En la segunda implementación el programa utiliza estructuras de control, funciones y estructuras de datos internas y externas para manejo de información en un dispositivo de almacenamiento externo (Google Drive) a través de un Menú Interactivo de opciones. Debe permitir realizar las operaciones básicas para manejo de archivos como crear, leer, actualizar o eliminar registros de estudiantes almacenados de manera persistente en un archivo binario dentro de Google Drive.



Resolución propuesta

A continuación se entrega una propuesta de solución (en formato imagen) para cada implementación que usted deberá analizar, codificar en Python y ejecutar para hacer un seguimiento de los resultados obtenidos. Puede que exista algún error que usted deberá corregir si al probarlo no funciona de acuerdo al enunciado propuesto.

Posible Solución Sugerida - Implementación 1

```
from tabulate import tabulate
import time

def mostrar_menu():
    # Muestra el menú de opciones
    print("\n=== Sistema de Gestión de Estudiantes ===")
    print("1. Agregar Estudiante (Alta)")
    print("2. Eliminar Estudiante (Baja)")
    print("3. Modificar Estudiante")
    print("4. Mostrar Todos los Estudiantes")
    print("5. Salir")
    print("-----")
    opcion = input("Seleccione una opción: ")
    return opcion

def agregar_estudiante(listadoEstudiantes):
    # Implementa la funcionalidad de Alta
    global ID_CONTADOR
    print("\n--- AGREGAR NUEVO ESTUDIANTE ---")
    nombre = input("Nombre completo: ")

    # Validar que la edad sea un número dentro de ciertos valores
    print("La edad debe estar entre 0 y 100 años.")
    edad = int(input("Edad: "))
    while edad <= 0 or edad >= 100:
        print("Entrada inválida. Por favor, ingrese un número para la edad.")
        print("La edad debe estar entre 0 y 100 años.")
        edad = int(input("Edad: "))

    # Validar que el promedio sea un número flotante
    promedio = float(input("Promedio (0.0 a 10.0): "))
    while promedio < 0.0 or promedio > 10.0:
        print("El promedio debe estar entre 0.0 y 10.0.")
        print("Entrada inválida. Por favor, ingrese un número para el promedio.")
        promedio = float(input("Promedio (0.0 a 10.0): "))

    # Crear estructura para información del estudiante
    nuevo_estudiante = {
        'id': ID_CONTADOR,
        'nombre': nombre,
        'edad': edad,
        'promedio': promedio
    }

    # Agregar estudiante
    listadoEstudiantes.append(nuevo_estudiante)
    print(f"\n✅ Estudiante '{nombre}' agregado con ID: {ID_CONTADOR}")
    ID_CONTADOR += 1
```

```
def eliminar_estudiante(listadoEstudiantes):
    # Implementa la funcionalidad de Baja
    id_a_eliminar = int(input("Ingrese el ID del estudiante a eliminar: "))

    seElimino = False

    # Buscar y verificar
    for estudiante in listadoEstudiantes:
        if estudiante['id'] == id_a_eliminar:
            print("Estudiante encontrado. Eliminando...")
            time.sleep(1)
            print("Tres")
            time.sleep(1)
            print("Dos")
            time.sleep(1)
            print("Uno")
            time.sleep(1)
            listadoEstudiantes.remove(estudiante)
            print(f"✅ Estudiante con ID {id_a_eliminar} y nombre '{estudiante['nombre']}'
eliminado.")
            seElimino = True
            break

    if not seElimino:
        print(f"⚠ Estudiante con ID {id_a_eliminar} no encontrado.")
```

```
def modificar_estudiante(lista):
    # Implementa la funcionalidad de Modificación
    print("\n--- MODIFICAR ESTUDIANTE ---")
    id_a_modificar = int(input("Ingrese el ID del estudiante a modificar: "))

    seModifico = False

    # Buscar el estudiante
    for estudiante in listadoEstudiantes:
        if estudiante['id'] == id_a_modificar:
            print(f"Estudiante encontrado: {estudiante['nombre']}")

            # Modificación de campos
            nueva_edad = input(f"Nueva Edad (actual: {estudiante['edad']}): ")
            estudiante['edad'] = nueva_edad

            # Modificación y validación del promedio
            nuevo_promedio = float(input(f"Nuevo Promedio (actual: {estudiante['promedio']}): "))
            while nuevo_promedio < 0.0 or nuevo_promedio > 10.0:
                print("Entrada inválida. El promedio debe estar entre 0.0 y 10.0. Intene nuevamente...1")
                nuevo_promedio = float(input(f"Nuevo Promedio (deje vacío para no cambiar, actual: {estudiante['promedio']}): "))
            estudiante['promedio'] = nuevo_promedio

            print(f"✅ Estudiante con ID {id_a_modificar} modificado exitosamente.")
            seModifico = True
    if not seModifico:
        print(f"⚠️ Estudiante con ID {id_a_modificar} no encontrado.")
```

```
def mostrar_estudiantes(listadoEstudiantes):
    # Muestra la lista de todos los estudiantes en formato tabla
    print("\n--- LISTA DE ESTUDIANTES ---")
    if not listadoEstudiantes:
        print("La lista de estudiantes está vacía.")
        return

    # Crea un diccionario para los encabezados de la tabla, asignando las claves del
    # diccionario a los nombres de encabezado deseados.
    headers_dict = {
        'id': 'ID',
        'nombre': 'NOMBRE',
        'edad': 'EDAD',
        'promedio': 'PROMEDIO'
    }

    # Muestra el listado de estudiantes en formato tabla
    print(tabulate(listadoEstudiantes, headers=headers_dict, tablefmt="fancy_grid"))

def ejecutar_programa():
    """Función principal que ejecuta el menú."""
    # Bucle principal del programa
    while True:
        opcion = mostrar_menu()

        #
        if opcion == '1':
            agregar_estudiante(listadoEstudiantes)
        elif opcion == '2':
            eliminar_estudiante(listadoEstudiantes)
        elif opcion == '3':
            modificar_estudiante(listadoEstudiantes)
        elif opcion == '4':
            mostrar_estudiantes(listadoEstudiantes)
        elif opcion == '5':
            print("👋 Saliendo del sistema. ¡Hasta pronto!")
            break
        else:
            print("❌ Opción no válida. Por favor, seleccione un número del 1 al 5.")

# Programa principal
listadoEstudiantes = []
ID_CONTADOR = 1
ejecutar_programa()
```

Posible Solución Sugerida - Implementación 2

```
from tabulate import tabulate
import time
import pickle
import os

from google.colab import drive
drive.mount('/gdrive/')

ruta_archivo = "/gdrive/MyDrive/estudiantes.dat"

# Crear estructura para información del estudiante (usada como registro)
estudiante = {
    'id': 0,
    'nombre': '',
    'edad': 0,
    'promedio': 0.0
}

def mostrar_menu():
    """Muestra el menú de opciones."""
    print("\n=== Sistema de Gestión de Estudiantes ===")
    print("1. Crear archivo de estudiantes")
    print("2. Agregar Estudiante (Alta)")
    print("3. Eliminar Estudiante (Baja)")
    print("4. Modificar Estudiante")
    print("5. Mostrar Todos los Estudiantes")
    print("6. Salir")
    print("-----")
    opcion = input("Seleccione una opción: ")
    return opcion

def crear_archivo_estudiantes():
    # Crea el archivo de estudiantes
    if not os.path.isfile(ruta_archivo):
        with open(ruta_archivo, 'wb') as archivo:
            print('Archivo de estudiantes creado con éxito')
    else:
        print('El archivo ya se encuentra creado')
        r = input("Desea crearlo nuevamente ? S/N: ")
        while r.upper() != 'S' and r.upper() != 'N':
            print('Respuesta invalida. Debe ingresar S para Si o N para No')
            r = input('Desea crear el archivo nuevamente ? S/N ')

        if r.upper() == 'S':
            with open(ruta_archivo, 'wb') as archivo:
                print('Archivo de estudiantes creado con éxito')
        else:
            print('Has elegido no crear nuevamente el archivo')
```

```
def guardar_datos(estudiante):
    # Graba los datos actuales de estudiante en el archivo binario en Google Drive
    with open(ruta_archivo, 'ab') as archivo:
        pickle.dump(estudiante, archivo)

def agregar_estudiante():
    # Implementa la funcionalidad de Alta
    global ID_CONTADOR
    print("\n--- AGREGAR NUEVO ESTUDIANTE ---")

    with open(ruta_archivo, 'ab') as archivo:
        nombre = input("Nombre completo: ")

        # Validar edad
        print("La edad debe estar entre 0 y 100 años.")
        edad = int(input("Edad: "))
        while edad <= 0 or edad >= 100:
            print("Entrada inválida. La edad debe estar entre 0 y 100.")
            edad = int(input("Edad: "))

        # Validar promedio
        promedio = float(input("Promedio (0.0 a 10.0): "))
        while promedio < 0.0 or promedio > 10.0:
            print("Entrada inválida. El promedio debe estar entre 0.0 y 10.0.")
            promedio = float(input("Promedio (0.0 a 10.0): "))

        estudiante = {
            "id": ID_CONTADOR,
            "nombre": nombre,
            "edad": edad,
            "promedio": promedio,
        }

        guardar_datos(estudiante)
        print(f"\n✅ Registro de Estudiante '{nombre}' agregado al archivo con ID: {ID_CONTADOR}")
        ID_CONTADOR += 1
```

```
def eliminar_estudiante():
    # Implementa la funcionalidad de Baja
    ruta_archivo_aux = "/gdrive/MyDrive/estudiantesAux.dat"

    seElimino = False

    with open(ruta_archivo, 'rb') as archivo:
        with open(ruta_archivo_aux, 'wb') as archivoAux:
            print("")
            id_a_eliminar = int(input("Ingrese el ID del estudiante a eliminar: "))

            t = os.path.getsize(ruta_archivo)
            while(archivo.tell() < t):
                estudiante = pickle.load(archivo)

                if estudiante["id"] == id_a_eliminar:
                    print(f"Registro de estudiante encontrado: {estudiante['nombre']}. Eliminando...")
                    time.sleep(1)
                    print("Tres")
                    time.sleep(1)
                    print("Dos")
                    time.sleep(1)
                    print("Uno")
                    time.sleep(1)
                    print(f"✅ Registro de Estudiante con ID {id_a_eliminar} eliminado.")
                    seElimino = True
                else:
                    pickle.dump(estudiante, archivoAux)

    if not seElimino:
        print(f"⚠️ Registro de Estudiante con ID {id_a_eliminar} no encontrado.")

    os.remove(ruta_archivo)
    os.rename(ruta_archivo_aux, ruta_archivo)
```

```
def modificar_estudiante():
    # Implementa la funcionalidad de Modificación
    print("\n--- MODIFICAR ESTUDIANTE ---")

    seModifico = False

    ruta_archivo_aux = "/gdrive/MyDrive/estudiantesAux.dat"

    with open(ruta_archivo, 'rb') as archivo:
        with open(ruta_archivo_aux, 'wb') as archivoAux:
            print("")
            id_a_modificar = int(input("Ingrese el ID del estudiante a modificar: "))

            t = os.path.getsize(ruta_archivo)
            while(archivo.tell() < t):
                estudiante = pickle.load(archivo)

                if estudiante["id"] == id_a_modificar:
                    print(f"Registro de estudiante encontrado: {estudiante['nombre']}")
                    print("Se le solicitaran los datos a modificar...")

                    # Modificación de campos
                    nueva_edad = input(f"Nueva Edad (actual: {estudiante['edad']}): ")
                    estudiante['edad'] = nueva_edad

                    # Modificación y validación del promedio
                    nuevo_promedio = float(input(f"Nuevo Promedio (actual: {estudiante['promedio']}): "))
                    while nuevo_promedio < 0.0 or nuevo_promedio > 10.0:
                        print("Entrada inválida. El promedio debe estar entre 0.0 y 10.0. Intene nuevamente...1")
                        nuevo_promedio = float(input(f"Nuevo Promedio (deje vacío para no cambiar, actual: {estudiante['promedio']}): "))
                    estudiante['promedio'] = nuevo_promedio

                    pickle.dump(estudiante, archivoAux)
                    seModifico = True
                    print(f"✅ Estudiante con ID {id_a_modificar} modificado exitosamente.")
                else:
                    # si son distintos, no se encontro el registro, grabar sin modificar
                    pickle.dump(estudiante, archivoAux)

    if not seModifico:
        print(f"⚠️ Registro de Estudiante con ID {id_a_modificar} no encontrado.")

    os.remove(ruta_archivo)
    os.rename(ruta_archivo_aux, ruta_archivo)
```

```
def mostrar_estudiantes():
    # Muestra la lista de todos los estudiantes
    print("\n--- LISTA DE ESTUDIANTES ---")

    with open(ruta_archivo, 'rb') as archivo:
        listadoEstudiantes = list()
        t = os.path.getsize(ruta_archivo)
        while(archivo.tell() < t):
            listadoEstudiantes.append(pickle.load(archivo))

    if not listadoEstudiantes:
        print("La lista de estudiantes está vacía.")
        return

    # Crea un diccionario para los encabezados, asignando las claves del diccionario a los nombres de
    encabezado deseados.
    headers_dict = {
        'id': 'ID',
        'nombre': 'NOMBRE',
        'edad': 'EDAD',
        'promedio': 'PROMEDIO'
    }

    print(tabulate(listadoEstudiantes, headers=headers_dict, tablefmt="fancy_grid"))


def ejecutar_programa():
    # Función principal que ejecuta el menú
    while True:
        opcion = mostrar_menu()

        if opcion == "1":
            crear_archivo_estudiantes()
        elif opcion == "2":
            agregar_estudiante()
        elif opcion == "3":
            eliminar_estudiante()
        elif opcion == "4":
            modificar_estudiante()
        elif opcion == "5":
            mostrar_estudiantes()
        elif opcion == "6":
            print("👋 Saliendo del sistema. ¡Hasta pronto!")
            break
        else:
            print("❌ Opción no válida. Por favor, seleccione un número del 1 al 5.")

#Programa Principal
ID_CONTADOR = 1
ejecutar_programa()
```